geeksforgeeks.org/problems/minimum-sum-partition3317/1Deadline

Search... Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Output Window

Compilation Results Custom Input

Compilation Completed

Case 1

Input:

arr[]=

1 6 5 11

Your Output:

1

Expected Output:

1

C++ (17) Start Timer

```cpp
        int totalSum = accumulate(arr.begin(), arr.end(), 0);

        // 2. The ideal target for one subset is half the total sum
        int target = totalSum / 2;

        // 3. dp[j] will be true if a subset with sum j is possible.
        // We only need to check up to the target sum.
        vector<bool> dp(target + 1, false);

        // A sum of 0 is always possible (empty subset)
        dp[0] = true;

        // 4. Populate the DP table
        for (int num : arr) {
            // Traverse backwards from target down to num.
            // Going backwards ensures we only use each element once per subset.
            for (int j = target; j >= num; --j) {
                dp[j] = dp[j] || dp[j - num];
            }
        }

        // 5. Find the largest possible subset sum (maxSubsetSum) that is <= target
        int maxSubsetSum = 0;
        for (int j = target; j >= 0; --j) {
            if (dp[j]) {
                maxSubsetSum = j;
                break;
            }
        }

        // 6. Calculate the result
        // If subset 1 has sum maxSubsetSum, subset 2 has sum (totalSum - maxSubsetSum).
        // The difference is: (totalSum - maxSubsetSum) - maxSubsetSum
        // = totalSum - 2 * maxSubsetSum
        return totalSum - 2 * maxSubsetSum;
    }
};
```

Ctrl + '

Custom Input Compile & Run Submit

geeksforgeeks.org/problems/minimum-sum-partition3317/1Deadline

Search... Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1**
	Accuracy : 100%

Points Scored	Time Taken
8 / 8	**0.72**
Your Total Score: 74 ↑	

Solve Next

Partition array to k subsets | Palindromic Partitioning | MaxSkill

Stay Ahead With:

C++ (17) Start Timer

```cpp
        int totalSum = accumulate(arr.begin(), arr.end(), 0);

        // 2. The ideal target for one subset is half the total sum
        int target = totalSum / 2;

        // 3. dp[j] will be true if a subset with sum j is possible.
        // We only need to check up to the target sum.
        vector<bool> dp(target + 1, false);

        // A sum of 0 is always possible (empty subset)
        dp[0] = true;

        // 4. Populate the DP table
        for (int num : arr) {
            // Traverse backwards from target down to num.
            // Going backwards ensures we only use each element once per subset.
            for (int j = target; j >= num; --j) {
                dp[j] = dp[j] || dp[j - num];
            }
        }

        // 5. Find the largest possible subset sum (maxSubsetSum) that is <= target
        int maxSubsetSum = 0;
        for (int j = target; j >= 0; --j) {
            if (dp[j]) {
                maxSubsetSum = j;
                break;
            }
        }

        // 6. Calculate the result
        // If subset 1 has sum maxSubsetSum, subset 2 has sum (totalSum - maxSubsetSum).
        // The difference is: (totalSum - maxSubsetSum) - maxSubsetSum
        // = totalSum - 2 * maxSubsetSum
        return totalSum - 2 * maxSubsetSum;
    }
};
```

Custom Input Compile & Run Submit